

W2 Soundness Requirements for zkVM Submissions

Ethereum Foundation Cryptography Team

2 June 2026

Purpose

The goal of W2 is not to provide the final end-to-end security proof for the zkVM. Instead, W2 should make each submission ready for such a proof.

When planning the W2 submission, the teams should choose one of several options to approach the full proof in W3:

- **from scratch:** describe the actual system in full details using third-party components (commitment schemes, hash functions, etc.) as black boxes, then prove the security using already established results for those components.
 - **vanilla scheme with deviations:** if the actual system uses seriously modified third-party components, then describe a simpler, *vanilla* system that uses unaltered primitives, establish a security argument for the vanilla system, and then extend it to the full construction through a sequence of soundness-preserving steps;
- and make necessary steps towards it in W2 (details in section 3).

In W2, each team should provide a structured description of its proof system that lets identify:

- the exact relations being proved;
- the vanilla construction against which the actual protocol should be compared;
- the differences between the vanilla construction and the actual protocol;

- the knowledge-soundness claims and assumptions needed for each proof-system component;
- the lemmas or proof obligations that remain for the final W3 security argument.

Required W2 Deliverables

0. Format

- The teams should use their W1 submission as the basis for the W2 submission.
- The W2 submission should be in a PDF format, with the source file (LaTeX/Markdown/Typx) provided alongside.

1. Recursion Topology

Describe the topology of all proof-system calls used by the zkVM.

This should include:

- base proofs;
- recursive proofs;
- aggregation proofs;
- continuations;
- any proof instances used for memory, lookup, permutation, range-check, or auxiliary arguments;
- the public inputs and outputs passed between proof instances.

The topology may be given as a diagram, a table, or a structured text description.

The main purpose is to make clear which proof system proves what statement, and how the statements compose.

2. Precise Relation for Each Proof-System Invocation

For every proof instance state the relation being proved.

This includes, but is not limited to:

- SNARK or STARK invocations;
- lookup arguments;
- memory or permutation arguments;
- range-check arguments;
- recursion verifier circuits;
- aggregation arguments;
- any auxiliary proof system whose soundness is needed by the zkVM.

For each instance, specify:

- public inputs;
- private witnesses;
- claimed statement;
- relevant constraints or predicates;
- how the proof is handled by the rest of the system.

3. Soundness Proof Sketch

Each team should provide a sketch of the soundness proof for the submitted protocol.

This may be:

- an informal proof;
- a chain of lemmas to be proved;
- a combination of standard references and new proof obligations.

The proof sketch should explain why an accepting proof implies that the stated relation holds, except with the claimed soundness error.

The proof sketch does not need to be fully formal at W2, but it should be precise enough that the remaining W3 proof obligations are visible.

Teams may write the proof sketch directly from scratch. Alternatively, they may use the following baseline-and-deviation strategy:

1. define a *vanilla* construction using well-known components whose security properties are already established;
2. sketch why the vanilla construction is sound;
3. clearly list all differences between the vanilla construction and the actual protocol;
4. argue that each difference preserves soundness;
5. if multiple differences interact across layers, explicitly describe the combined soundness argument or state the lemma that will be needed for W3.

The vanilla construction does not need to be efficient and does not need to match the production implementation exactly. Its role is to provide a baseline proof strategy against which the actual protocol can be compared.

The vanilla construction may include the following components:

- plain AIR constraints;
- a standard PIOP or IOP;
- plain FRI;
- a standard wrapper proof system used for recursion, aggregation, or compression.

Only proof-system-level differences are required. Pure implementation optimizations are out of scope unless they change the mathematical statement being proved, the transcript, the constraints, the commitment scheme, the low-degree test, the lookup argument, recursion, aggregation, or any other soundness-relevant component.

For each difference, state:

- what is changed;
- which layer it affects;
- why the change is believed to preserve soundness;
- whether it interacts with other changes.

Placement of Auxiliary Arguments

Lookup, memory, permutation, and range-check arguments should not be hidden only inside the AIR or only inside the PIOP. Each such argument proves its own auxiliary relation and may have its own PIOP, committed polynomials, transcript challenges, and PCS/FRI openings.

For each such component, teams should state:

- the relation it proves;
- the (vanilla) argument used as baseline;
- the PIOP or algebraic reduction used;
- the committed polynomials and degree bounds;
- the PCS/FRI openings used;
- the transcript challenges used;
- the soundness claim;
- the deviations from the vanilla version.

Expected Level of Detail

W2 should be sufficient for an external reviewer to answer the following questions.

- What is the exact relation each proof-system invocation proves?
- What is the clean vanilla proof system for this relation?
- How does the actual protocol differ from the vanilla construction?
- Which differences are proof-system-level changes?
- What soundness theorem is expected for the vanilla construction?

- Which lemmas are standard, adapted from literature, or new?
- What remains to be proved in W3?

W2 does not require:

- a final end-to-end zkVM security proof;
- a full knowledge-soundness proof;
- formal verification of the implementation;
- detailed documentation of pure implementation optimizations that do not affect the proof system.